

Documentation Technique

Voici le contenu du code source :

binary_converter.c

Gère la sauvegarde et le chargement des quadrees au format binaire.

Fonctions principales

- **save_c_tree_binary** : Sauvegarde un quadtree couleur dans un fichier binaire.
- **save_bw_tree_binary** : Sauvegarde un quadtree noir et blanc dans un fichier binaire.
- **load_c_tree_binary** : Charge un quadtree couleur depuis un fichier binaire.
- **load_bw_tree_binary** : Charge un quadtree noir et blanc depuis un fichier binaire.

c_tree.c

Gestion des arbres colorés.

Fonctions principales

- **create_c_node** : Crée un noeud avec les enfants donnés.
- **create_c_leaf** : Crée une feuille avec la couleur donnée.
- **display_c_tree** : Affiche l'arbre.
- **c_exact_distance** : Mesure la distance exacte entre deux arbres.
- **c_mean_distance** : Mesure la distance moyenne entre deux arbres.
- **mean_color** : Retourne la couleur moyenne des couleurs des noeuds enfants donnés.
- **free_c_leaf** : Libère la mémoire allouée pour la feuille.
- **free_c_tree** : Libère la mémoire allouée pour l'arbre t.

bw_tree.c

Gestion des arbres en noir et blanc.

Fonctions principales

- `create_bw_node` : Crée un noeud avec les données et les enfants donnés.
- `create_bw_leaf` : Crée une feuille.
- `display_bw_tree` : Affiche les noeuds de l'arbre `bw_tree`.
- `bw_distance` : Mesure la distance entre deux arbres, suppose `t1` et `t2` non NULL.
- `free_bw_tree` : Libère la mémoire allouée pour l'arbre `t`.

color.c

Gestion des couleurs.

Fonctions principales

- `create_color` : Crée une couleur.
- `print_color` : Affiche la couleur `c`.
- `c_c_distance` : Calcule la distance entre deux couleurs.
- `c_distance` : Retourne la distance d'une seule couleur par rapport à l'origine.
- `c_equals` : Teste si deux couleurs sont égales.
- `free_color` : Libère la mémoire allouée pour la couleur `c`.

gui.c

Gestion de l'interface graphique.

Fonctions principales

- `initialize_window` : Initialise la fenêtre graphique.
- `draw_buttons` : Dessine les boutons pour différentes fonctionnalités.

- **handle_buttons** : Gère les clics sur les boutons et effectue les actions correspondantes.
- **average_color** : Calcule la couleur moyenne d'une zone rectangulaire dans une image.
- **create_c_tree_from_image** : Crée un quadtree (*c_tree*) à partir d'une image.
- **approximate_image** : Approxime une image en utilisant un quadtree.

minimizer.c

Les minimisations des quadtrees (Avec et Sans pertes).

Fonctions principales

- **create_hash_table** : Crée une table de hachage.
- **free_hash_table** : Libère la mémoire allouée pour la table de hachage.
- **minimize_identical_leaves_in_node** : Minimise un quadtree en combinant les feuilles identiques.
- **minimize_unique_leaves** : Minimise un quadtree en gardant une seule feuille par couleur, stocke les feuilles dans la table de hachage donnée.

Architecture :

- **binary_converter.c / binary_converter.h**:

Fonctionnalité: Sauvegarde et charge un quadtree au format binaire.

Fonctions clés:

save_c_tree_binary(const char *filename*, *c_node* tree): Sauvegarde un quadtree couleur dans un fichier binaire.

save_bw_tree_binary(const char *filename*, *bw_node* bw_tree): Sauvegarde un quadtree noir et blanc dans un fichier binaire.

load_c_tree_binary(const char *filename*): Charge un quadtree couleur depuis un fichier binaire.

load_bw_tree_binary(const char *filename*): Charge un quadtree noir et blanc depuis un fichier binaire.

save_c_node(BitBuffer *bit_buffer*, *c_node* node): Fonction auxiliaire pour parcourir et sérialiser les noeuds colorés.

save_bw_node(BitBuffer *bit_buffer*, *bw_node* node): Fonction auxiliaire pour parcourir et sérialiser les noeuds en noir et blanc.

load_c_node(BitBuffer *bit_buffer*): Fonction auxiliaire pour parcourir et désérialiser les noeuds colorés.

load_bw_node(BitBuffer bit_buffer): Fonction auxiliaire pour parcourir et désérialiser les noeuds en noir et blanc.

- bw_tree.c / bw_tree.h:

Fonctionnalité: Manipule des arbres en noir et blanc.

Fonctions clés:

*create_bw_node(char data, bw_node **children)*: Crée un noeud avec des enfants.

create_bw_leaf(): Crée une feuille.

display_bw_tree(bw_node t): Affiche l'arbre en noir et blanc.

bw_distance(bw_node t1, bw_node t2): Calcule la distance entre deux arbres.

free_bw_tree(bw_node t): Libère la mémoire allouée pour l'arbre.

- c_tree.c / c_tree.h:

Fonctionnalité: Manipule des arbres colorés.

Fonctions clés:

create_c_node(c_node children): Crée un noeud avec des enfants.

create_c_leaf(color color): Crée une feuille avec la couleur donnée.

display_c_tree(c_node t): Affiche l'arbre coloré.

c_exact_distance(c_node t1, c_node t2): Calcule la distance exacte entre deux arbres.

c_mean_distance(c_node t1, c_node t2): Calcule la distance moyenne entre deux arbres.

mean_color(c_node children): Calcule la couleur moyenne des enfants.

free_c_tree(c_node *t): Libère la mémoire allouée pour l'arbre.

- color.c / color.h:

Fonctionnalité: Manipule des couleurs.

Fonctions clés:

create_color(unsigned char red, unsigned char green, unsigned char blue, unsigned char alpha): Crée une couleur.

print_color(color c): Affiche une couleur.

c_c_distance(color c1, color c2): Calcule la distance entre deux couleurs.

c_distance(color c1): Calcule la distance d'une couleur par rapport à l'origine.

c_equals(const color c1, const color c2): Teste si deux couleurs sont égales.

*free_color(color *c)*: Libère la mémoire allouée pour une couleur.

- gui.c / gui.h:

Fonctionnalité: Gère l'interface graphique pour l'application de compression d'image.

Fonctions clés:

initialize_window(): Initialise la fenêtre graphique.

draw_buttons(): Dessine les boutons de l'interface.

`handle_buttons(MLV_Image image, c_node tree)`: Gère les clics sur les boutons et les actions correspondantes.
`average_color(MLV_Image image, int x, int y, int width, int height)`: Calcule la couleur moyenne d'une zone d'une image.
`create_c_tree_from_image(MLV_Image image, int x, int y, int length)`: Crée un quadtree à partir d'une image.
`draw_c_tree_as_image(c_node c_tree, char shape)`: Dessine le quadtree comme une image.
`approximate_image(MLV_Image image, char shape)`: Approxime une image en utilisant un quadtree.

- `minimizer.c / minimizer.h`:

Fonctionnalité: Minimise un quadtree en combinant des feuilles identiques.

Fonction clé:

`create_hash_table()`: Crée une table de hachage.

`free_hash_table(HashTable hash_table)`: Libère la mémoire de la table de hachage.

`minimize_identical_leaves_in_node(c_node node)`: Minimise les noeuds du quadtree en combinant des feuilles identiques.

`minimize_unique_leaves(c_node node, HashTable hash_table)`: Minimise un quadtree en gardant une seule feuille par couleur, et stocke les feuilles dans la table de hachage.

- `main.c`:

Fonctionnalité: Point d'entrée de l'application.

Fonctionnalité clé:

Initialisation de la fenêtre graphique.

Gestion des boutons et des actions utilisateur.

- `makefile`:

Fonctionnalité: Automatisation de la compilation. Cibles clés:

`all`: Compile le programme principal.

`debug`: Compile le programme avec les informations de débogage.

`clean`: Supprime les fichiers compilés.

minimisation (simple 1):

Elle est conçue pour minimiser un quadtree en combinant des feuilles identiques. Elle fonctionne en parcourant de manière récursive tous les noeuds du quadtree et en vérifiant si les enfants d'un noeud donné sont tous des feuilles avec la même couleur. Si c'est le cas, les enfants sont supprimés et remplacés par une seule feuille représentant cette couleur. Cette approche réduit la taille de l'arbre en éliminant les redondances de couleur.

minimisation (simple 2):

Table de hachage : Nous avons défini une structure `HashTable` pour stocker les feuilles uniques par couleur. La fonction `hash_color` calcule une clé de hachage pour une couleur donnée.

Fonctions de la table de hachage :

- `create_hash_table()`: Initialise la table de hachage.
- `free_hash_table()`: Libère la mémoire de la table de hachage.
- `find_leaf()`: Recherche une feuille dans la table de hachage.
- `add_leaf()`: Ajoute une feuille dans la table de hachage.

Minimisation :

- `minimize_unique_leaves(c_node node, HashTable hash_table)`: Fonction principale qui crée la table de hachage et appelle la fonction auxiliaire pour traverser l'arbre et minimiser les feuilles.